PhisTools

A Policy-Aware Phishing Risk Analysis and Prevention Support Tool

Technical Project Report - Network Information Security (NIS)

Project Positioning

PhisTools is presented as an NIS-oriented phishing triage and risk-assessment prototype. Machine learning provides predictive evidence, while the main value comes from security analysis: header and spoofing checks, suspicious URL analysis, policy-aware verdicting, attack-type classification, logging, and incident-style response guidance.

Prepared for	Network Information Security (NIS) course submission
Prepared by	Team members: Devansh Agarwal (23BIT0028) Nakul Thombare (23BIT0013) Samarth Khandelwal (23BIT0051)
Faculty / Guide	Dr Shantharajah S P
Academic Year	2025-2026

Abstract. PhisTools is a Streamlit-based phishing assessment system that accepts email subject/body content, optional headers, URLs, and an optional remote attachment URL. The system combines email and URL model probabilities with Network Information Security indicators to produce a final risk score, threat level, verdict, attack-type classification, and prevention or response recommendations. The project is intentionally framed as a security triage workflow rather than a standalone classifier. Its educational value lies in demonstrating how machine learning, heuristic security analysis, policy thresholds, logging, and explainability can be integrated into one analyst-facing prototype.

Executive Summary

The project addresses phishing as a multi-signal security problem. Instead of relying only on text classification, PhisTools combines email language analysis, URL analysis, header anomaly checks, risk scoring, attack categorization, and action recommendations. This makes it more suitable for NIS demonstration because the final output resembles a lightweight analyst-support tool.

Primary problem	Phishing campaigns that combine social engineering, spoofed headers, suspicious URLs, and occasionally malicious attachments.
Primary deliverable	A Streamlit web application that performs policy-aware phishing risk analysis and presents explainable results.
Core runtime file	webapp.py
Main strengths	Combined security reasoning, explainable outputs, policy modes, logging, attack-type classification, and response advice.
Main limitations	URL calibration remains imperfect for some trusted domains; remote attachment scanning depends on remote server behavior; full RFC-grade email forensics are out of scope.

Submission Note

For evaluation, this project should be judged as a phishing risk-analysis prototype for NIS. Its strongest value is not a single probability score, but the way it turns multiple technical indicators into a structured, explainable security verdict.

1. Introduction and Problem Statement

Phishing remains one of the most common attack vectors in institutional and enterprise environments. Modern phishing attempts do not rely on only one signal: attackers often combine urgent language, lookalike domains, reply-to manipulation, brand impersonation, and deceptive links. A single-signal detector may therefore miss context, or it may produce a label without explaining the associated security implications.

PhisTools was developed to bridge this gap. The project aims to combine machine-learning predictions with Network Information Security logic so that the output is useful for triage and education. Instead of ending at "phishing / not phishing," the system explains why the content is risky, classifies the likely attack style, and recommends prevention or incident-response actions.

2. Objectives and Scope

The project was designed around one primary objective and several supporting objectives.

- Build a practical phishing analysis tool that accepts email content, optional headers, and URLs.

- Combine predictive ML signals with security indicators such as spoofing hints, suspicious domains, and urgency cues.
- Generate a final risk score and policy-aware verdict rather than a raw probability alone.
- Provide attack-type classification and prevention or response recommendations.
- Maintain scan history and feedback records for review.

The current scope is prototype-oriented. The system is not intended to replace an enterprise secure email gateway, perform complete SPF/DKIM/DMARC validation, or detonate files inside a malware sandbox. Instead, it demonstrates how a compact NIS workflow can be assembled from multiple technical components.

3. System Overview

From the user perspective, the system behaves as a single-interface phishing triage application. A user provides suspicious email content, optional header text, optional URLs, and optionally a remote attachment URL. The application then processes those inputs through machine-learning models and security heuristics before presenting a structured result.

Intended users	Students, instructors, and small teams performing phishing-awareness or NIS demonstration exercises.
Inputs	Subject text, email body text, optional raw headers, one or more URLs, optional remote attachment URL.
Outputs	Email model probabilities, URL probabilities, header risk, final risk score, threat level, verdict, attack-type categories, recommended actions, grouped reasons, logs, and optional feedback.
High-level workflow	Input collection -> model inference -> NIS feature extraction -> risk fusion -> verdicting -> explanation -> logging.

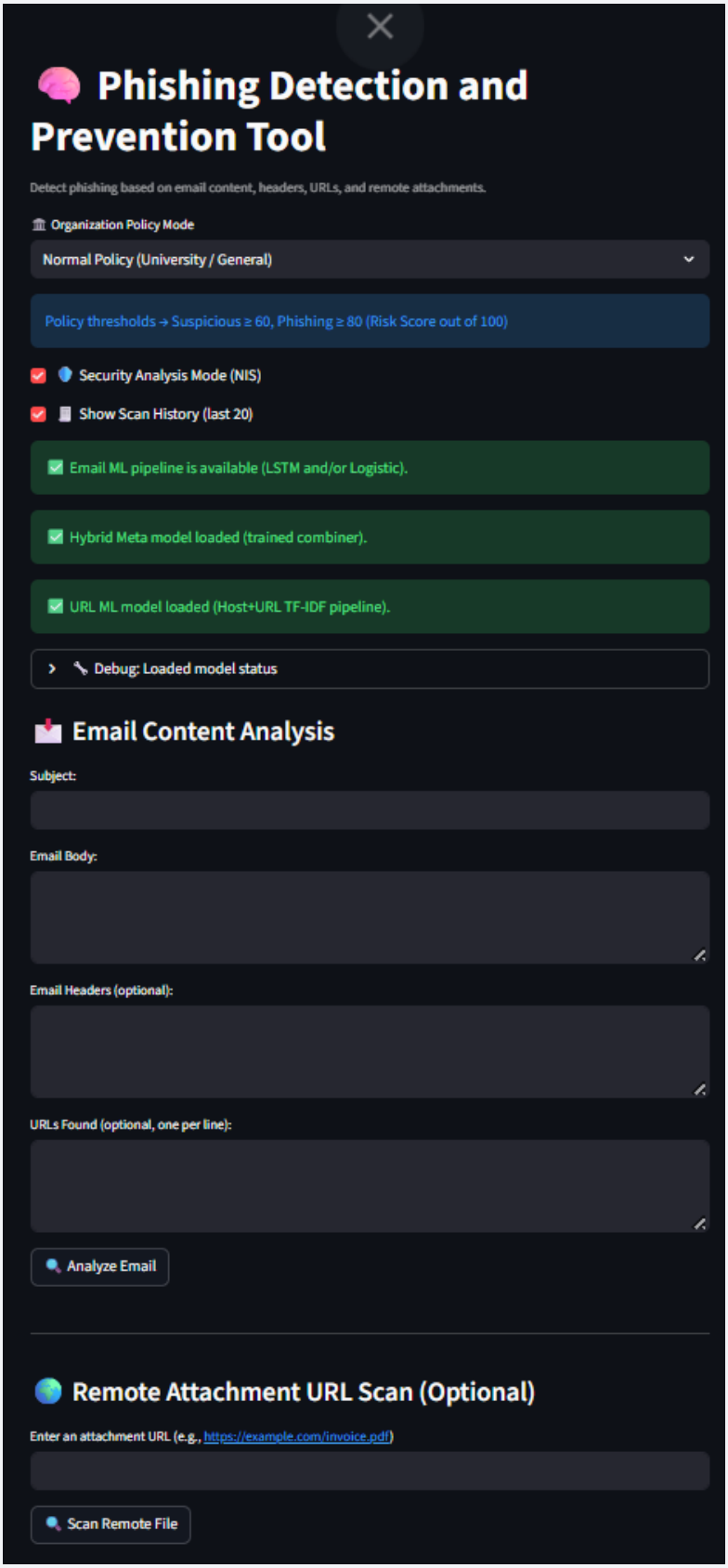


Figure 1. Main Streamlit interface of PhisTools showing policy selection, input controls, and analysis actions.

4. Runtime Architecture and Working Flow

The active entrypoint is webapp.py. At startup, the application sets compatibility options for Keras model loading, imports helper modules from utils/, and loads the required runtime artifacts from models/. The Streamlit interface then renders input fields, policy mode controls, and optional analysis toggles.

- Email analysis path: logistic regression, LSTM, and optional hybrid meta model are used to estimate phishing probability from subject and body text.
- Header analysis path: rules inspect reply-to, return-path, and other inconsistencies that may indicate spoofing or sender manipulation.
- URL analysis path: a runtime URL model scores each URL, and heuristic features contribute to NIS interpretation.
- Risk fusion path: the application combines ML outputs, heuristics, offline intelligence, and policy thresholds to compute a 0-100 risk score and a final verdict.
- Response path: attack types and recommended actions are generated so the output resembles analyst-support guidance.
- Operational path: scan history and feedback are written to CSV files for review.

Current runtime-critical components
webapp.py orchestrates the system. The most important supporting modules are utils/nis_features.py, utils/risk_scoring.py, utils/scan_logger.py, utils/threat_intel.py, and utils/attachment_file_analyzer.py. The runtime model artifacts include the email logistic/vectorizer pair, the LSTM with tokenizer, the optional hybrid meta model, the email metadata file, and the active URL model artifact.

5. Positioning the Project: NIS versus ML

A key requirement for this report is to present the project primarily as an NIS submission rather than as an isolated machine-learning exercise. This positioning is justified by the way the application is structured.

Project layer	What it contains	Why it matters
ML layer	Email logistic model, email LSTM, optional hybrid meta model, URL model.	Provides probabilistic evidence about phishing-like text and suspicious URLs.
NIS layer	Header and spoofing checks, suspicious URL/domain indicators, policy thresholds, attack categorization, recommended actions, scan logging, optional intelligence	Turns raw evidence into security interpretation and triage guidance.

	checks.	
Hybrid layer	Final risk score, verdict, grouped explainability, policy-aware escalation.	Combines ML and security logic into a practical analyst-style output.

In simple terms, the ML models answer "how phishing-like does this content appear?" The NIS logic answers "how risky is this event from a security perspective, what kind of attack is it, and what should the user or team do next?" This distinction is what makes PhisTools suitable for a Network Information Security course.

6. Technology Stack and Dependencies

The implementation is Python-based. Streamlit is used for the frontend and interactive runtime. Scikit-learn provides the logistic, TF-IDF, pipeline, and calibration components. TensorFlow/Keras is used for the LSTM model. Pandas and NumPy support data handling, while requests and python-magic support remote fetching and file-type analysis for optional attachment scanning.

Language	Python
UI framework	Streamlit
Email ML stack	TF-IDF + Logistic Regression, LSTM, optional hybrid meta model
URL ML stack	Canonicalized URL/host text pipeline with structural feature support and probability calibration
Security utilities	Header/spoofing checks, NIS feature extraction, risk scoring, threat-intel helpers, scan logging
Datasets	CEAS_08.csv for email path; merged URL datasets generated into data/merged_urls.csv for URL path
Known dependency note	requirements.txt historically required updates to fully match active runtime dependencies.

7. Important Files and Their Roles

The repository contains runtime, retraining, and experimental files. For clarity, they should be read in three groups.

7.1 Runtime-critical files

webapp.py	Active entrypoint; orchestrates UI rendering, model loading,

	inference, risk fusion, verdicting, and logging.
utils/nis_features.py	Extracts email and URL security indicators used in risk scoring and explainability.
utils/risk_scoring.py	Computes the base risk score and reason set used by the final verdict layer.
utils/scan_logger.py	Persists scan history to logs/scan_logs.csv.
utils/threat_intel.py	Provides offline intelligence and optional online lookups when API keys are available.
utils/attachment_file_analyzer.py	Performs MIME and file-based heuristics for downloaded remote attachments.
models/*	Stores the runtime artifacts loaded at startup, including the email models, tokenizer files, metadata, and active URL model.

7.2 Retraining files

train_logistic_model.py	Generates the logistic email model and preferred TF-IDF vectorizer.
train_deep_model.py	Generates the LSTM email model, tokenizer files, and email metadata.
train_hybrid_model.py	Generates the hybrid meta model that fuses logistic and LSTM outputs.
train_all.py	Integrated script for generating the main email-model stack.
train_url_ml_model.py	Produces the active runtime URL model artifact.
merge_datasets.py	Prepares the merged URL dataset consumed by train_url_ml_model.py.

7.3 Experimental or alternate files

A small number of training or utility files exist for earlier experiments, visual training runs, or toy models. They are useful for project history but are not required for normal application execution. This explains why the repository contains multiple train scripts even though webapp.py is the only runtime entrypoint.

8. Implemented Features

Feature	Purpose	Type	Runtime status

Email ML scoring	Computes logistic, LSTM, and fused probabilities from email content.	ML / Hybrid	Active
URL scoring	Scores each URL and supports per-link plus average risk display.	ML + NIS context	Active
Header analysis	Checks for spoofing-style anomalies such as mismatched reply paths.	NIS	Active
Risk scoring	Combines evidence into a final 0-100 security score.	Hybrid	Active
Policy mode	Adjusts suspicious and phishing thresholds by organizational posture.	NIS	Active
Attack typing	Maps signal patterns to categories such as credential harvesting or spoofing.	NIS	Active
Recommendations	Provides prevention and response actions based on threat level and attack type.	NIS	Active
Logging and feedback	Stores scans and feedback for review and iteration.	Operational	Active
Remote attachment scan	Optionally downloads and inspects a remote file URL.	NIS operational	Optional
Threat-intel checks	Adds offline intelligence and optional API-	NIS enrichment	Optional

backed external checks.

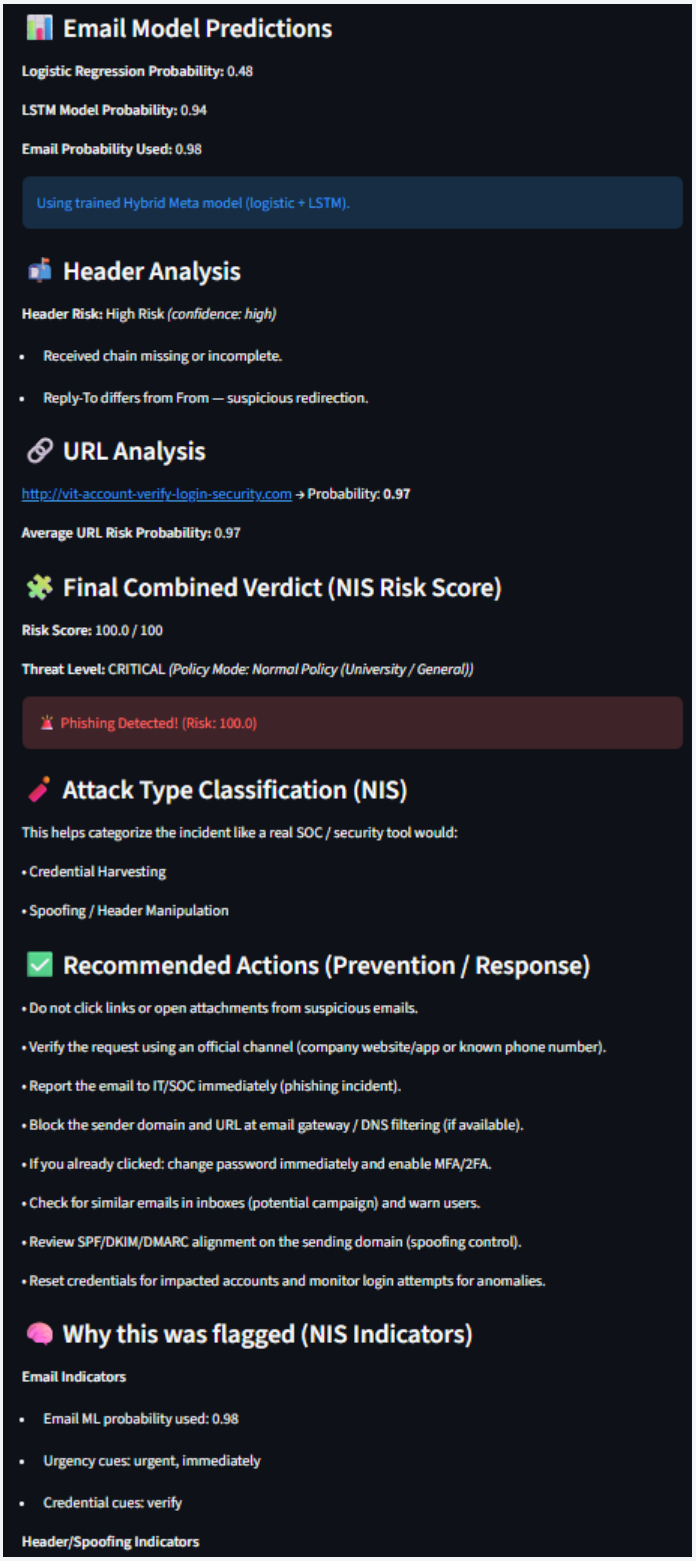


Figure 2. High-confidence phishing detection with attack categorization and response guidance.

9. Models, Artifacts, and Fallback Behavior

The email path uses three related runtime artifacts: a logistic regression model with TF-IDF vectorizer, an LSTM model with tokenizer files, and an optional hybrid meta model. The hybrid learns how to combine the logistic and LSTM probabilities, allowing a more stable final email probability when both signals are available.

The URL path uses the active runtime artifact models/url_tfidf_logistic_model.pkl. This artifact is consumed directly by webapp.py. The application also uses fallback behavior: if one or more models fail to load, the UI remains operational and degrades to available evidence or neutral scoring.

Email logistic artifact	models/phish_logistic_model.pkl
Email TF-IDF vectorizer	models/phish_tfidf_vectorizer.pkl
Email LSTM artifact	models/phish_lstm_model.h5
Tokenizer files	models/tokenizer.json and/or models/tokenizer.pkl
Hybrid meta artifact	models/hybrid_model.pkl
Email metadata	models/email_meta.json
Active URL artifact	models/url_tfidf_logistic_model.pkl

10. Dataset and Training Pipeline

The project uses CEAS_08.csv for email-oriented training and a merged URL dataset generated from phishing URL sources for the URL branch. The merge_datasets.py script standardizes URL and label columns before train_url_ml_model.py is used to produce the active URL runtime artifact.

Recent URL retraining work added structural URL features in addition to the previous lexical representation. Examples include host length, URL length, subdomain depth, digit ratio, hyphen count, suspicious keyword count, IP-host flags, suspicious TLD flags, punycode indicators, path depth, and shortener flags. Probability calibration was also introduced to reduce false positives, while preserving the same runtime artifact name and interface.

Training-script interpretation

None of the train scripts are executed during normal use of webapp.py. They exist to build or rebuild the saved model artifacts. A smaller subset is core for regeneration of runtime models; others are experimental or older variants retained for reference.

11. Testing and Observed Behavior

Manual testing was carried out using safe emails, urgency-driven phishing text, credential-harvesting scenarios, brand impersonation, shortened URLs, spoofing/header mismatch cases, multiple-URL cases, and partial-input cases. The observations below summarize the behavior seen during testing.

- The system performs well when multiple malicious signals align, such as urgency language, spoofed headers, and fake login URLs.
- Attack-type classification and recommended actions are among the strongest parts of the output because they make the result practically useful.
- Header-related outputs are helpful for NIS demonstration, especially when reply-to or return-path mismatches are present.
- The URL branch historically over-flagged some trusted domains; retraining reduced false positives overall, but calibration is still not perfect for every trusted enterprise domain.
- Partial or manually typed headers may still lead to less reliable header interpretation than full raw header dumps.
- Remote attachment analysis remains optional and network-dependent; some sites may return 403 or otherwise block automated fetch attempts.

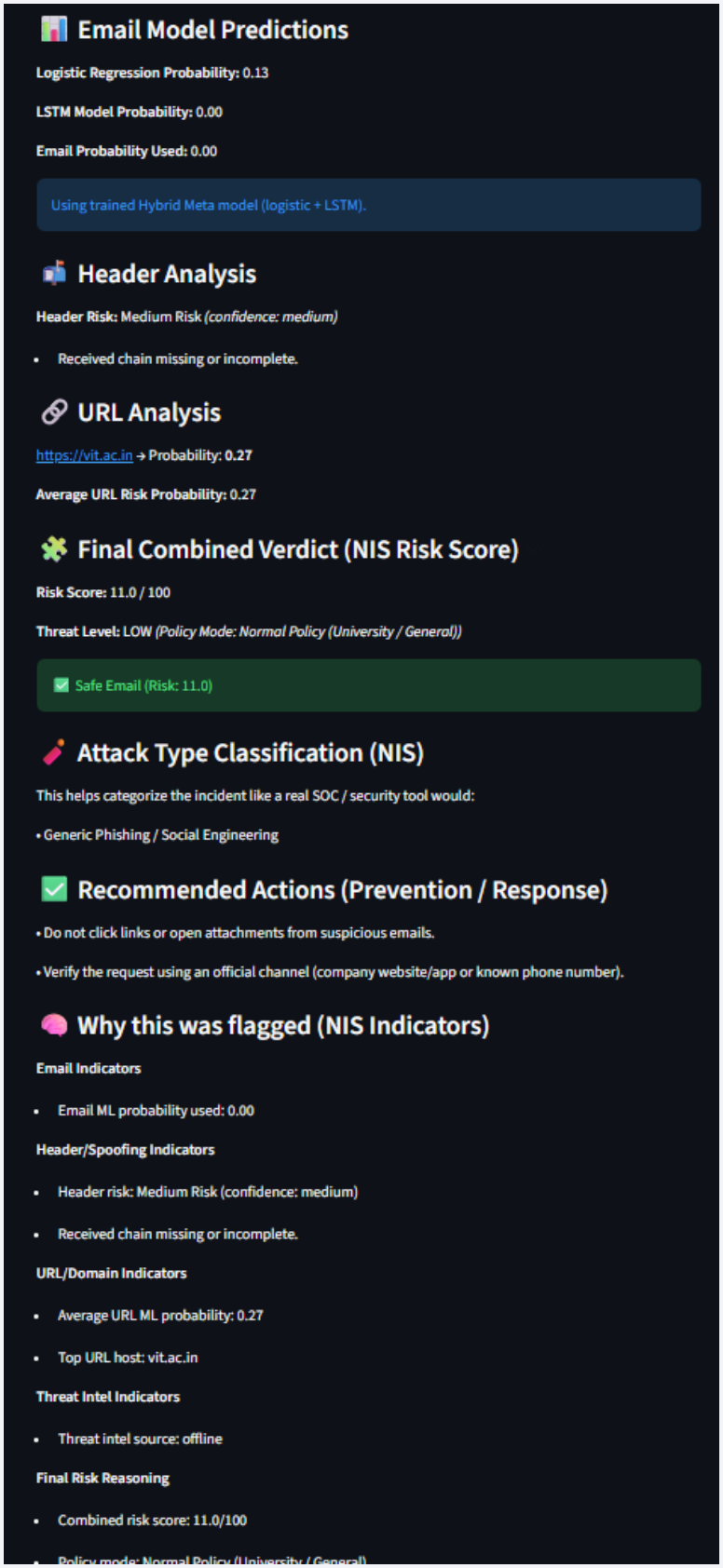


Figure 3. Example safe-case analysis with low threat level and grouped explanatory indicators.

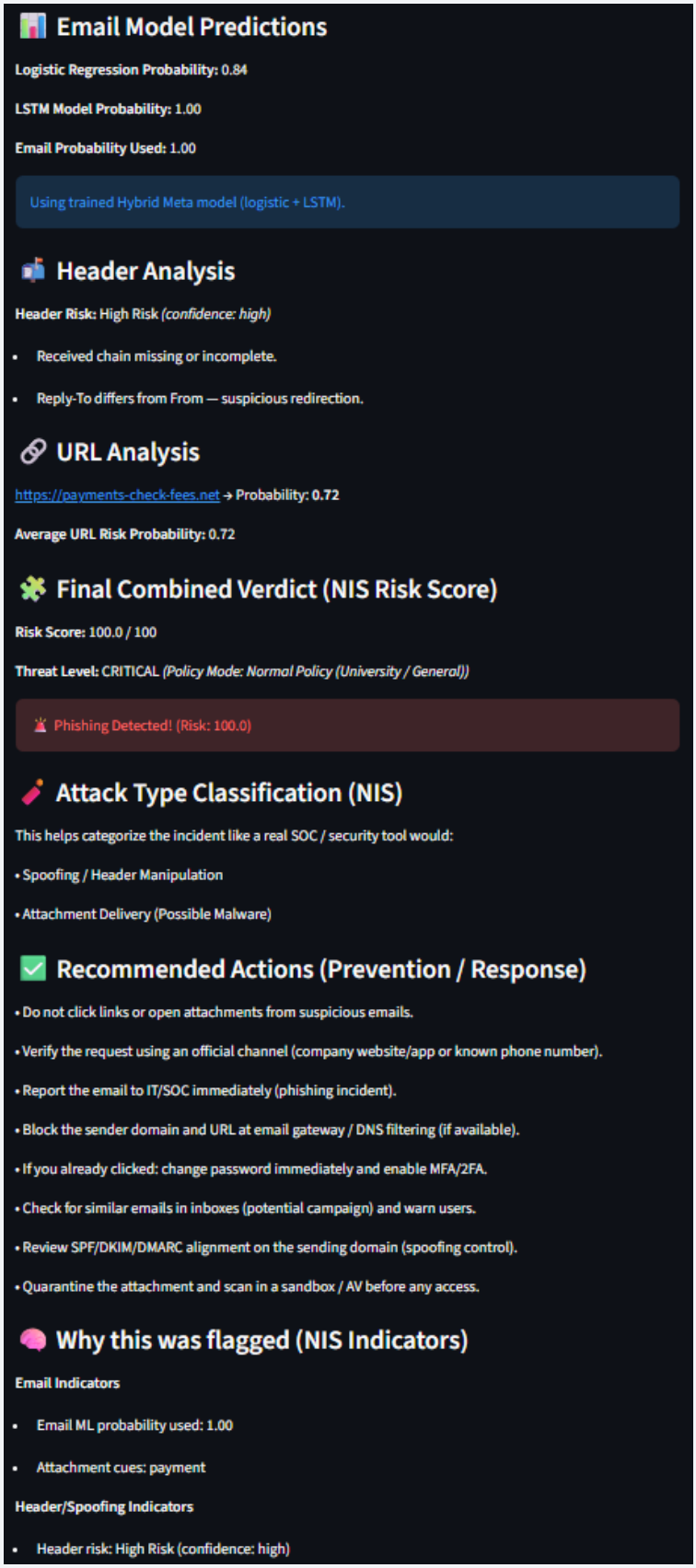


Figure 4. Header-spoofing indicator output demonstrating mismatch-based escalation.

12. Improvements Made During Iteration

The project was refined after runtime testing. The improvement work focused on reliability, calibration, and clarity rather than large-scale redesign. This was important because the app already worked as a prototype, but some outputs were too harsh or too difficult to explain cleanly during a demo.

- Header analysis was improved by introducing partial or manual-header detection and confidence-aware scoring, reducing unjustified inflation from missing fields alone.
- Post-score calibration was added so that weak evidence contributes less aggressively, while combined strong signals such as suspicious URLs plus spoofing cues escalate more appropriately.
- Explainability was grouped into analysis categories such as Email Indicators, Header/Spoofing Indicators, URL/Domain Indicators, Threat Intel Indicators, and Final Risk Reasoning.
- Remote attachment scanning was made more robust with safer validation, clearer failure messaging, redirect and timeout handling, and better cleanup behavior.
- The URL training pipeline was upgraded with structural features, calibration, and benign-domain anchoring to reduce holdout false positives while preserving malicious separation.

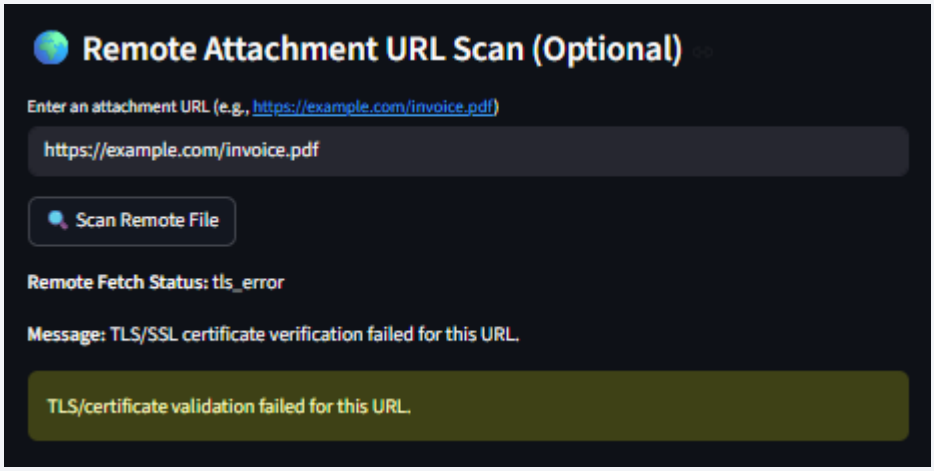


Figure 5. Remote attachment scan status classification with user-friendly success or failure messaging.

13. Limitations and Honest Assessment

PhisTools is a meaningful academic prototype, but it is not yet production-hardened. Its limitations should be stated clearly because acknowledging them strengthens the credibility of the submission.

- The URL model still exhibits imperfect calibration for certain trusted domains and should not be treated as a sole verdict source.
- Header analysis depends heavily on the completeness and realism of the provided headers; full RFC-grade forensic parsing is not implemented.
- Optional threat-intel checks depend on network access and configured API keys.

- Remote attachment scanning depends on external host policy, TLS conditions, and whether the provided link is a true downloadable file.
- Dependency reproducibility can still be affected if the runtime environment does not fully match active imports.

Most accurate project identity

PhisTools should be described as a phishing risk-analysis and prevention-support prototype. It is more than a classifier because it integrates policy, explainability, and security response guidance. At the same time, it should not be oversold as a production mail-security platform.

14. Novelty and Distinctive Value

What makes the project distinctive is not any single algorithm, but the way multiple components have been integrated. The system combines model-based evidence with security heuristics, grouped reasoning, policy-aware verdicting, operational logging, and action recommendations. This moves the prototype closer to a lightweight analyst workflow than a one-screen classifier demonstration.

15. What the Team Learned

The project provided learning at both the machine-learning and security-engineering levels. From the ML side, the team worked with logistic regression, tokenization, deep learning, model fusion, and probability calibration. From the NIS side, the team explored spoofing indicators, suspicious URL structure, risk scoring, explainability, and response planning. Equally important were engineering lessons: model compatibility, runtime robustness, logging, UI design, and the challenge of balancing false positives against useful security sensitivity.

16. Future Work

- Curate a larger benign URL corpus, especially for academic, enterprise, and government domains, to further reduce false positives.
- Introduce richer header parsing and optional SPF/DKIM/DMARC verification chains where feasible.
- Improve domain-reputation fusion and allowlist handling without making the tool overly rigid.
- Expand remote attachment analysis with safer host compatibility handling or sandbox-style workflows.
- Harden dependency management and packaging for more reproducible deployment.

Appendix A. Screenshot and Figure Plan

The final submission should include the following screenshots or result figures with formal captions.

Figure placeholder	What to capture	Suggested caption
Figure 1	Main UI with policy mode, inputs, and toggles visible.	Main Streamlit interface of PhisTools showing policy selection and input controls.
Figure 2	Phishing result showing high score, verdict, attack types, and actions.	High-confidence phishing detection with attack categorization and response guidance.
Figure 3	Safe or low-risk case with grouped reasons.	Example safe-case analysis with low threat level and explanatory indicators.
Figure 4	Header mismatch or spoofing case.	Header-spoofing indicator output demonstrating mismatch-based escalation.
Figure 5	Remote attachment success or failure message.	Remote attachment scan status classification with user-friendly result messaging.

Conclusion

PhisTools demonstrates how an educational phishing prototype can be elevated from a simple classifier into a more credible Network Information Security workflow. Its main contribution is the combination of ML evidence with security heuristics, policy-based scoring, attack categorization, logging, and response guidance. Although calibration and deployment hardening remain future tasks, the project is already meaningful as a NIS project and a practical learning artifact.